MATRIX CHAIN MULTIPLICATION

Module 2- DP problems

Recalling Matrix Multiplication

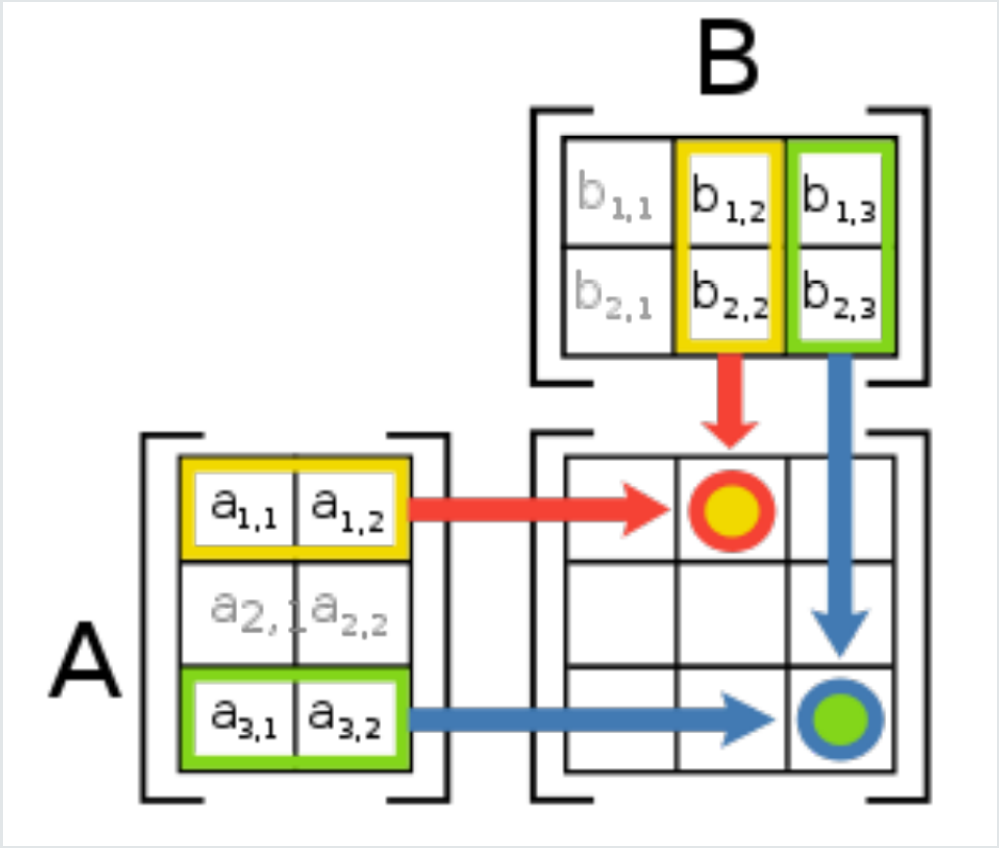
The product $C = AB$ of a $p \times q$ matrix A and a $q \times r$ matrix B is a $p \times r$ matrix given by

$$c[i,j] = \sum_{k=1}^{q} a[i,k]b[k,j]$$

for $1 \leq i \leq p$ and $1 \leq j \leq r$.

Example: If

$$A = \begin{bmatrix} 1 & 8 & 9 \\ 7 & 6 & -1 \\ 5 & 5 & 6 \end{bmatrix}, \qquad B = \begin{bmatrix} 1 & 8 \\ 7 & 6 \\ 5 & 5 \end{bmatrix},$$



$$C = AB = \begin{bmatrix} 102 & 101 \\ 44 & 87 \\ 70 & 100 \end{bmatrix}.$$

- **There are p . r total entries in C and each takes O(q) time to compute, thus the total time to multiply these two matrices is dominated by the number of scalar multiplication, which is p . q . r.**
- In other words, C has 'pr' entries, and each entry takes 'q' time to compute so the total procedure takes 'pqr'

Given a $p \times q$ matrix A, a $q \times r$ matrix B and a $r \times s$ matrix C, then ABC can be computed in two ways $(AB)C$ and $A(BC)$:

The number of multiplications needed are:

$$\begin{aligned} mult[(AB)C] &= pqr + prs, \\ mult[A(BC)] &= qrs + pqs. \end{aligned}$$

When $p = 5$, $q = 4$, $r = 6$ and $s = 2$, then

$$\begin{aligned} mult[(AB)C] &= 180, \\ mult[A(BC)] &= 88. \end{aligned}$$

A big difference!

Implication: The multiplication "sequence" (parenthesization) matters

Suppose we have 4 matrices:

A: 30 x 1

B: 1 x 40

C: 40 x 10

D: 10 x 25

((AB)(CD)) : requires 41,200 scalar multiplications

(A((BC)D)) : requires 1400 scalar multiplications

Example: Three matrices A_1, A_2, A_3 with dimensions

$$A_1 : 10 \times 100 \quad A_2 : 100 \times 5 \quad A_3 : 5 \times 50$$

$(p_0 = 10, p_1 = 100, p_2 = 5, p_3 = 50)$

Computation of $(A_1 \times A_2) \times A_3$:

- $A_1 \times A_2 = A_{12}$ requires $10 \cdot 100 \cdot 5 = 5000$ multiplications
- $A_{12} \times A_3$ requires $10 \cdot 5 \cdot 50 = 2500$ multiplications
- Total: 7500 multiplications

Computation of $A_1 \times (A_2 \times A_3)$:

- $A_2 \times A_3 = A_{23}$ requires $100 \cdot 5 \cdot 50 = 25000$ multiplications
- $A_1 \times A_{23}$ requires $10 \cdot 100 \cdot 50 = 50000$ multiplications
- Total: 75000 multiplications

THE PROBLEM

Given a sequence of matrices $A_1, A_2, A_3, ..., A_n$, find the best way (using the minimal number of multiplications) to compute their product.

- Isn't there only one way? $((\cdots((A_1 \cdot A_2) \cdot A_3)\cdots) \cdot A_n)$
- No, matrix multiplication is *associative*.
  e.g. $A_1 \cdot (A_2 \cdot (A_3 \cdot (\cdots (A_{n-1} \cdot A_n)\cdots)))$ yields the same matrix.
- Different multiplication orders do not cost the same:
  - Multiplying $p \times q$ matrix A and $q \times r$ matrix B takes $p \cdot q \cdot r$ multiplications; result is a $p \times r$ matrix.
  - Consider multiplying 10×100 matrix A_1 with 100×5 matrix A_2 and 5×50 matrix A_3.
    - $(A_1 \cdot A_2) \cdot A_3$ takes $10 \cdot 100 \cdot 5 + 10 \cdot 5 \cdot 50 = 7500$ multiplications.
    - $A_1 \cdot (A_2 \cdot A_3)$ takes $100 \cdot 5 \cdot 50 + 10 \cdot 50 \cdot 100 = 75000$ multiplications.

Notation

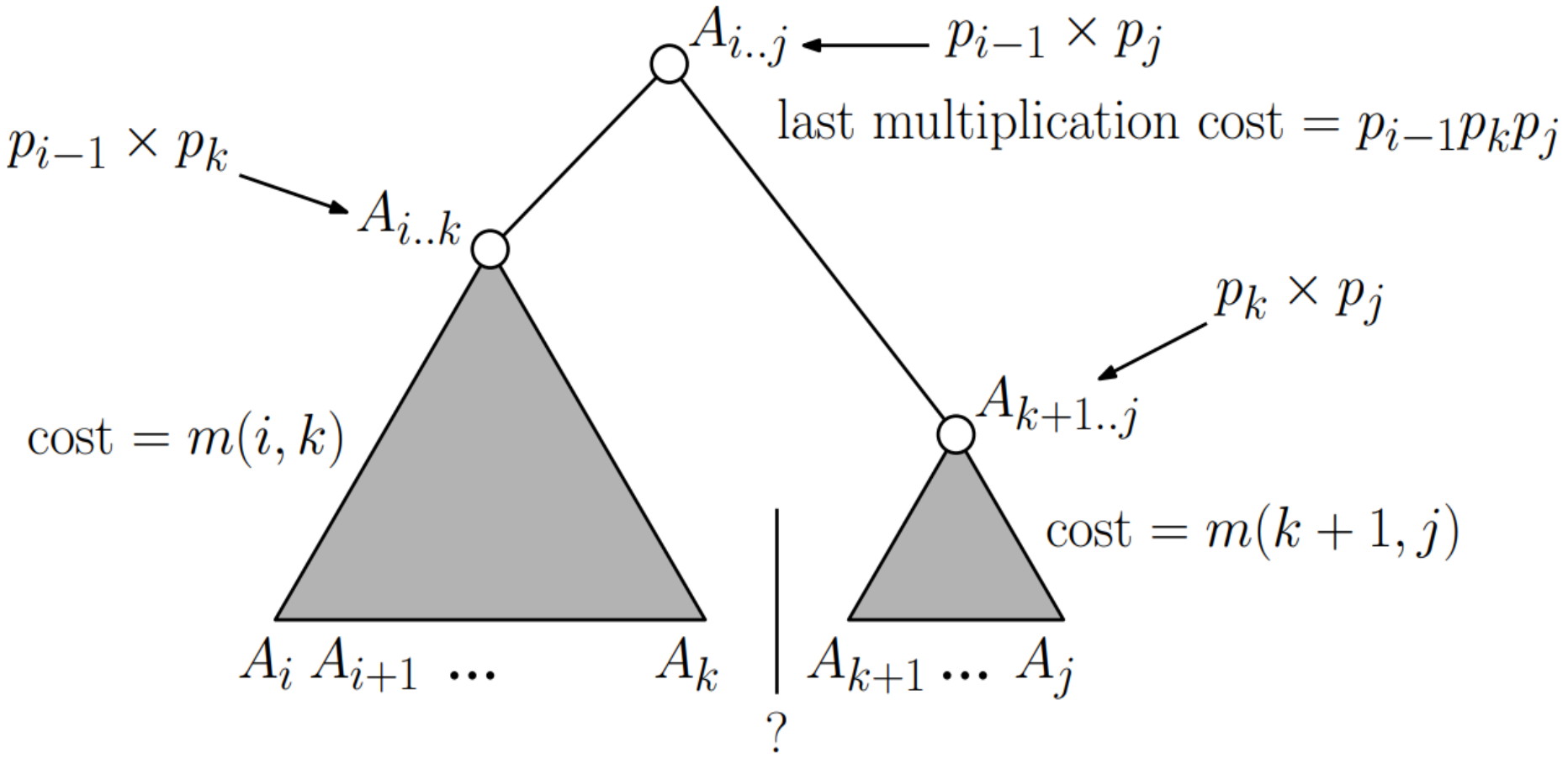
- In general, let A_i be $p_{i-1} \times p_i$ matrix.
- Let $m(i, j)$ denote minimal number of multiplications needed to compute $A_i \cdot A_{i+1} \cdots A_j$
- We want to compute $m(1, n)$.

Recursive algorithm

- Assume that someone tells us the position of the **last** product, say k. Then we have to compute recursively the best way to multiply the chain from i to k, and from $k+1$ to j, and add the cost of the final product. This means that

$$m(i, j) = m(i, k) + m(k + 1, j) + p_{i-1} \cdot p_k \cdot p_j$$

- If noone tells us k, then we have to try all possible values of k and pick the best solution.



- Recursive formulation of $m(i,j)$:

$$m(i,j) = \begin{cases} 0 & \text{If } i = j \\ \min_{i \le k < j}\{m(i,k) + m(k+1,j) + p_{i-1} \cdot p_k \cdot p_j\} & \text{If } i < j \end{cases}$$

- To go from the recursive formulation above to a program is pretty straightforward:

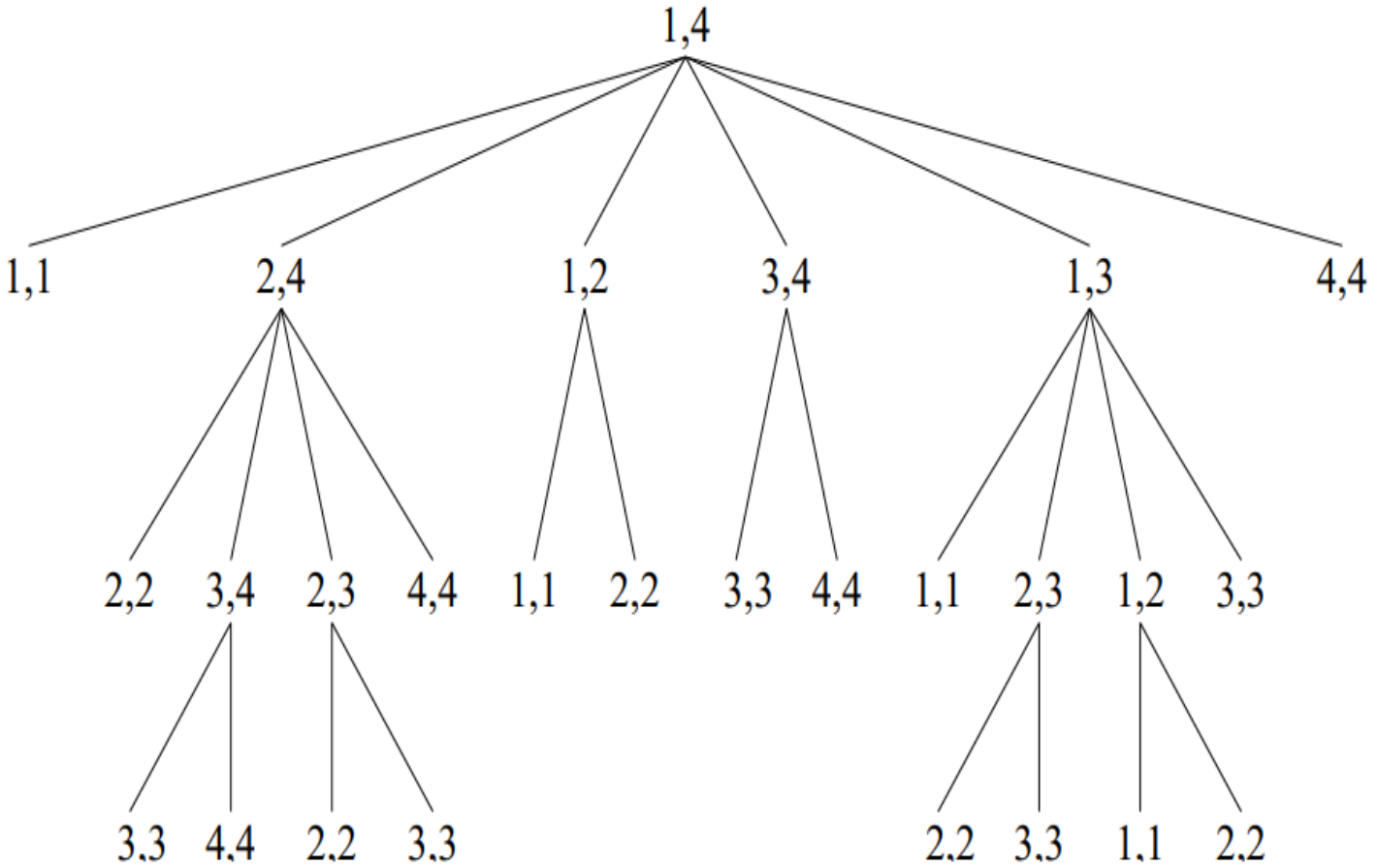
```
MATRIX-CHAIN(i, j)
    IF i = j THEN return 0
    m = ∞
    FOR k = i TO j − 1 DO
        q = MATRIX-CHAIN(i, k) + MATRIX-CHAIN(k + 1, j) + p_{i−1} · p_k · p_j
        IF q < m THEN m = q
    OD
    Return m
END MATRIX-CHAIN

Return MATRIX-CHAIN(1, n)
```

- Running time:

$$\begin{aligned}
T(n) &= \sum_{k=1}^{n-1} (T(k) + T(n-k) + O(1)) \\
&= 2 \cdot \sum_{k=1}^{n-1} T(k) + O(n) \\
&\geq 2 \cdot T(n-1) \\
&\geq 2 \cdot 2 \cdot T(n-2) \\
&\geq 2 \cdot 2 \cdot 2 \ldots \\
&= 2^n
\end{aligned}$$

- Exponential is ... SLOW!

- Problem is that we compute the same result over and over again.
  - Example: Recursion tree for MATRIX-CHAIN$(1, 4)$



Dynamic programming with a table and recursion

- Solution is to "remember" the values we have already computed in a table. This is

```
MATRIX-CHAIN(i, j)
    IF T[i][j] < ∞ THEN return T[i][j]
    IF i = j THEN T[i][j] = 0, return 0
    m = ∞
    FOR k = i to j − 1 DO
        q = MATRIX-CHAIN(i, k) + MATRIX-CHAIN(k + 1, j)+p_{i−1} · p_k · p_j
        IF q < m THEN m = q
    OD
    T[i][j] = m
    return m
END MATRIX-CHAIN

return MATRIX-CHAIN(1, n)
```

- The table will prevent a subproblem MATRIX-CHAIN(i,j) to be computed more than once

- Running time:
  - $\Theta(n^2)$ different calls to MATRIX-CHAIN(i, j).
  - The first time a call is made it takes $O(n)$ time, *not* counting recursive calls.
  - When a call has been made once it costs $O(1)$ time to make it again.

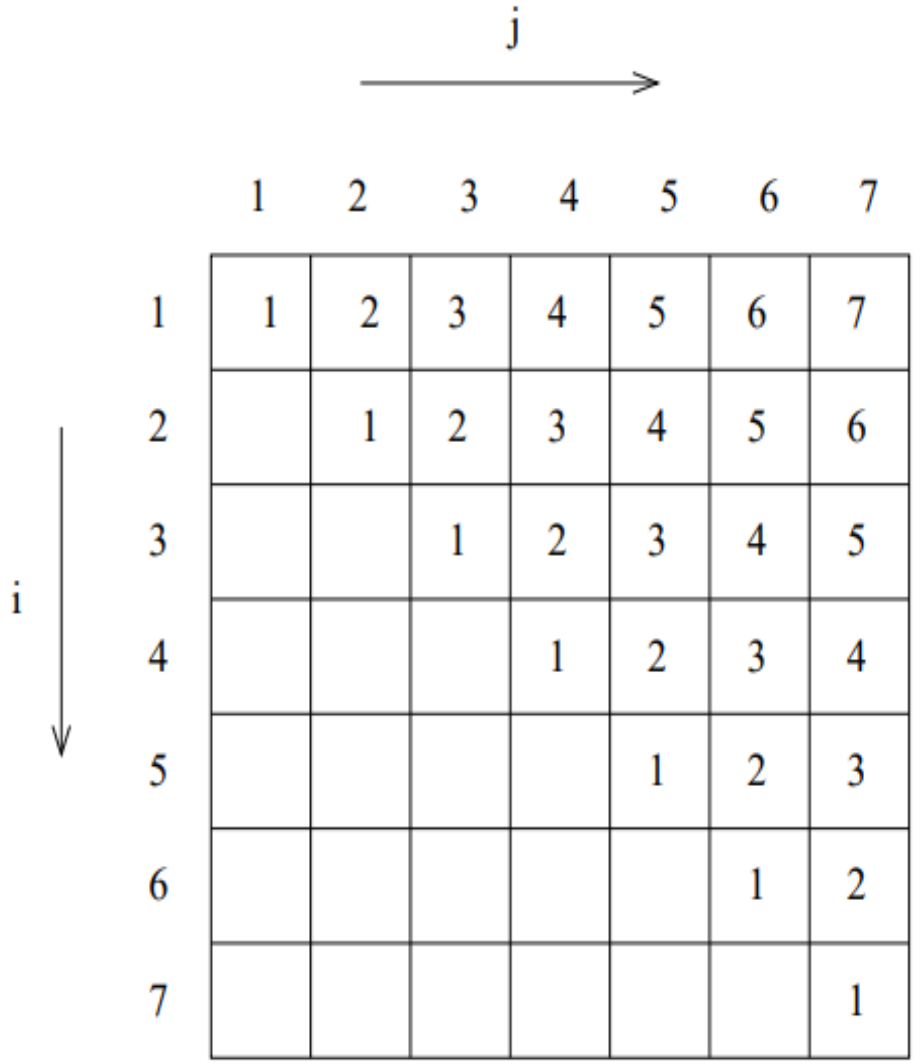
    $\Downarrow$

    $O(n^3)$ time
  - Another way of thinking about it: $\Theta(n^2)$ total entries to fill, it takes $O(n)$ to fill one.

Dynamic Programming without recursion

- Solution for Matrix-chain without recursion: Key is that $m(i,j)$ only depends on $m(i,k)$ and $m(k+1,j)$ where $i \leq k < j \Rightarrow$ if we have computed them, we can compute $m(i,j)$
  - We can easily compute $m(i,i)$ for all $1 \leq i \leq n$ $(m(i,i) = 0)$
  - Then we can easily compute $m(i,i+1)$ for all $1 \leq i \leq n-1$
    $m(i,i+1) = m(i,i) + m(i+1,i+1) + p_{i-1} \cdot p_i \cdot p_{i+1}$
  - Then we can compute $m(i,i+2)$ for all $1 \leq i \leq n-2$
    $m(i,i+2) = \min\{m(i,i) + m(i+1,i+2) + p_{i-1} \cdot p_i \cdot p_{i+2}, m(i,i+1) + m(i+2,i+2) + p_{i-1} \cdot p_{i+1} \cdot p_{i+2}\}$
    $\vdots$
  - Until we compute $m(1,n)$

Computation order:



– Computation order

FOR $i = 1$ to n DO

$T[i][i] = 0$

OD

FOR $l = 1$ to $n - 1$ DO

FOR $i = 1$ to $n - l$ DO

$j = i + l$

$T[i][j] = \infty$

FOR $k =$ i to $j - 1$ DO

$q = T[i][k] + T[k + 1][j] + p_{i-1} \cdot p_k \cdot p_j$

IF $q < T[i[j]$ THEN $T[i][j] = q$, $s[i, j] \leftarrow k$

OD

OD

OD

PRINT-OPTIMAL-PARENS (s, i, j)

```
1. if i=j
2. then print "A"
3. else print "("
4. PRINT-OPTIMAL-PARENS (s, i, s [i, j])
5. PRINT-OPTIMAL-PARENS (s, s [i, j] + 1, j)
6. print ")"
```

Complexity: The loops are nested three deep.

Each loop index takes on $\leq n$ values.

Hence the time complexity is $O(n^3)$. Space complexity $\Theta(n^2)$.